

Optimisation de contraintes distribuée

Aurélie Beynier

CoCoMa, Master 2 AI2D

octobre 2025

Plan

- 1 Contexte et Modèles
- 2 Résolution
 - ADOPT
 - DPOP
 - DSA
 - Travaux Pratiques
- 3 Perspectives
- 4 Exercices

DCOP

- DCOP : Distributed Constraint Optimization Problem
- Raisonnement basé sur les contraintes
- Chaque agent contrôle un ensemble de variables. Un agent ne contrôle que ses variables.
- Il existe des contraintes entre les valeurs des variables.
- Les agents doivent respecter ces contraintes et ne sont informés que des contraintes qui les impliquent.
- Le non respect des contraintes a un coût.
- Les agents doivent prendre des décisions locales (valeurs de leurs variables) afin de minimiser un coût global.
- Le modèle considère des agents coopératifs

DCOP

Pourquoi des contraintes ?

- Permet de modéliser des problématiques de coopération, de conflits, d'interaction
- Naturel dans de nombreux domaines : ordonnancement, allocation de ressources, planification

Exemples de problèmes

- Planification de rencontres (réunions) [RM04]
- Coloration de graphes [PM05]
- Réseaux de capteurs et Tracking [RM04]
- UAV, robotique

Modèle

Étant donné

- Un ensemble de variables $V = \{x_1, x_2, \dots, x_n\}$ chacune assignée à un agent
- Un ensemble de domaines discrets $\{D_1, D_2, \dots, D_n\}$ où D_1 correspond au domaine de définition de x_1
- Un ensemble de fonctions de coûts $f_{ij} : D_i \times D_j \rightarrow \mathbb{N}$ décrivant les objectifs des agents

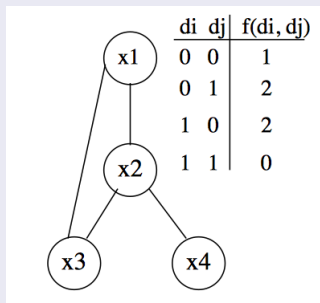
Objectif

- Assigner à chaque variable x_i une valeur dans son domaine D_i ($\mathcal{A}$ désigne l'allocation globale des valeurs)
- de manière à ce que la somme des coûts soit minimisée :

$$F(\mathcal{A}) = \sum_{x_i, x_j \in V} f_{ij}(d_i, d_j) \text{ tel que } x_i \leftarrow d_i, x_j \leftarrow d_j \in \mathcal{A} \quad (1)$$

Modèle

Exemple [PM05]



$$F(\{(x_1, 0), (x_2, 0), (x_3, 0), (x_4, 0)\}) = 4$$

$$F(\{(x_1, 1), (x_2, 1), (x_3, 1), (x_4, 1)\}) = 0$$

$$A^* = \{(x_1, 1), (x_2, 1), (x_3, 1), (x_4, 1)\}$$

Distribution et asynchronisme

- Chaque agent décide de la valeur de sa variable
- Les agents sont actifs simultanément
- Chaque agent a des informations privées
- Les agents peuvent communiquer entre eux mais cela a un coût

Contexte

- Les agents communiquent en s'envoyant des messages
- La communication n'est pas instantanée et prend un temps fini aléatoire
- Pas de perte de messages
- Les messages sont délivrés dans leur ordre d'envoi (pour chaque paire d'agents)
- Les contraintes sont binaires : n'impliquent pas plus de 2 agents
- Les agents ne connaissent que les contraintes qui les impliquent
- Chaque agent est associé à une seule variable (nb d'agents = nb de variables)

Contexte

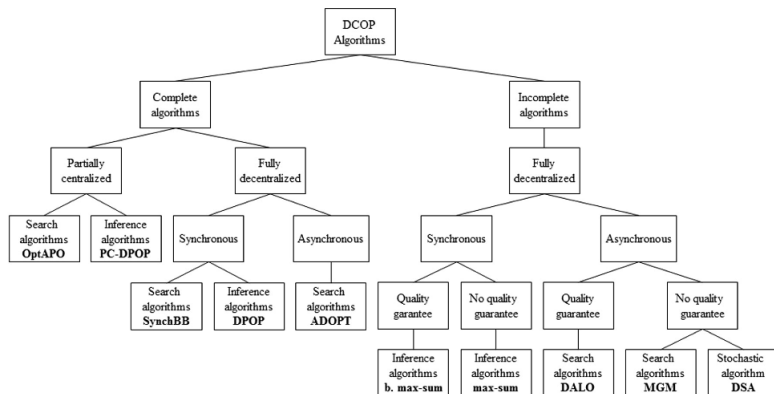
Différence avec les DEC-POMDPs :

- Pas de décision séquentielle
- Pas d'incertitude sur l'exécution des actions : il s'agit ici d'assigner une valeur à une variable
- Mais prise en compte de l'asynchronisme et des contraintes de communication

Complexité

- Complexité NP-hard
- Algorithmes optimaux vs approchés
- Algorithmes distribués vs partiellement distribués
- Asynchrones vs synchrones

Classification des algorithmes [LEB14]



ADOPT

Asynchronous Distributed Optimization - ADOPT [PM05]

- Premier algorithme distribué, optimal et asynchrone
- Communication locale et asynchrone : un agent communique par envois de messages à ses voisins et n'attend pas la réponse pour continuer la résolution.
- Les agents sont organisés par une structure d'arbre qui permet de définir des priorités entre les agents
- Utilisation de bornes inférieures : on backtrack quand la valeur d'une borne inférieure devient trop haute (et non quand la sous-optimalité est prouvée) → *weak backtracking*
- Résolution en mémoire limitée

ADOPT

Ordonnancement des agents par un pseudo-arbre

- Ordonnancement des agents par un Depth-First Search (DFS) Tree
- 1 seule racine, parents et enfants sont des voisins dans le graphe de contraintes
- Chaque agent a 1 seul parent et plusieurs enfants
- Pas de contraintes entre frères et sœurs
- Plusieurs arbres sont possibles pour un même graphe de contraintes
- Tout graphe de contraintes peut être arrangé sous forme d'un DFS
- Idée : en raison de la relative indépendance des nœuds situés dans les différentes branches, il est possible de résoudre ces branches en parallèle.

ADOPT

Ordonnancement des agents par un pseudo-arbre

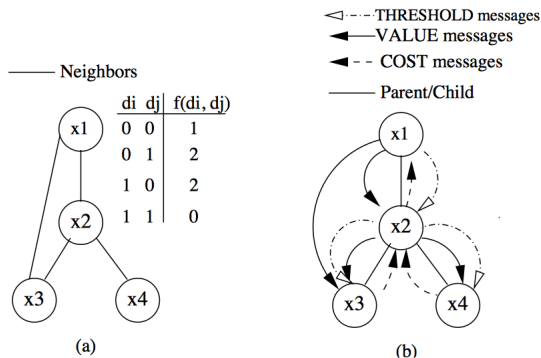


Figure 2: (a) Constraint graph. (b) Adopt communication graph.

ADOPT

Informations maintenues par chaque agent

- Un contexte courant : les valeurs prises par les agents de plus haute priorité (ascendants dans l'arbre)
- Pour chaque valeur que l'agent peut prendre et chaque fils :
 - borne inférieure (initialisée à 0 par exemple)
 - borne supérieure (initialisée à ∞)
 - threshold

ADOPT

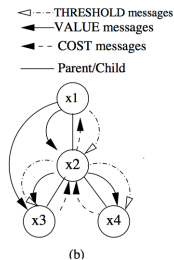
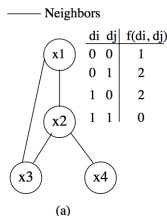
Best first search strategy

- Chaque agent choisit la meilleure valeur étant données ses informations locales courantes : valeur ayant la plus petite borne inférieure
- Une borne inf. est calculée initialement à partir d'un coût local puis mise à jour à partir des informations reçues de manière asynchrone

ADOPT

Communication asynchrone

- 3 types de messages : VALUE, COST, THRESHOLD
- Les messages VALUE sont envoyés aux descendants
- Les messages COST sont envoyés au parent : context, lower bound, upper bound
- Les messages THRESHOLD sont envoyés aux enfants



ADOPT

Threshold

Lorsqu'un agent sait que lb est une borne inf. pour son sous-arbre, il l'envoie aux enfants pour les en informer. Ainsi, ils ne cherchent pas une solution dont le coût est inférieur à lb

Value

Chaque agent mémorise les valeurs assignées aux variables par ses parents de plus haute priorité → solution partielle appelée *contexte*

Cost

Un message de coût est constitué d'un contexte, une borne inf. et une borne sup.

ADOPT

Calcul des coûts

Lorsqu'un agent reçoit des messages VALUE des parents, il en déduit un contexte courant à partir duquel il peut calculer un coût local (coût avec ses voisins).

$$\delta(d_i) = \sum_{x_j, d_j \in \text{CurrentContext}} f_{ij}(d_i, d_j) \quad (2)$$

$\delta(d_i)$ est le coût de choisir d_i pour x_i et j est un parent de i
Lorsqu'un agent reçoit des messages COST, il met à jour ses bornes inf. ($lb(d, x_l)$) pour calculer une borne inf. sur d :

$$LB(d_i) = \delta(d_i) + \sum_{x_l \in \text{Children}} lb(d_i, x_l) \quad (3)$$

De même pour les bornes supérieures.

ADOPT

Calcul des coûts

On en déduit les bornes inf. et sup :

$$LB = \min_{d \in D_i} LB(d) \quad (4)$$

De même pour les bornes sup.

Ces bornes sont envoyées aux parents.

Elles servent de condition de terminaison : $LB = UB$

ADOPT

Choix de la meilleure valeur

Pour chaque valeur d_I , calculer :

- $\delta(d_i)$: coût avec les ascendants de choisir d_i dans le contexte courant
- la borne inf. : $LB(d_i) = \delta(d_i) + \sum_{x_l \in \text{Children}} lb(d_i, x_l)$

On sélectionne alors la valeur d_i ayant la borne inférieure la plus petite et on communique la borne inf $LB(d_i)$ aux parents du nœud

ADOPT

Algorithme asynchrone

- Chaque fois qu'un agent reçoit un message :
 - Le traiter (changement de valeur possible)
 - Modifier le contexte courant
 - Mise à jour des bornes inférieure et supérieure
 - La valeur prise par l'agent est celle de borne inf. minimale
 - Envoyer la valeur aux descendants
 - Envoyer le coût aux parents

ADOPT termine en retournant une solution optimale :

$$LB = UB = Opt(x_i, CurrentContext)$$

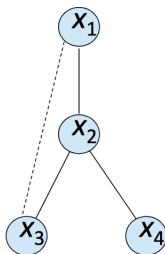
ADOPT : exemple

4 Variables (4 agents) x_1, x_2, x_3, x_4 with $D = \{a, b\}$

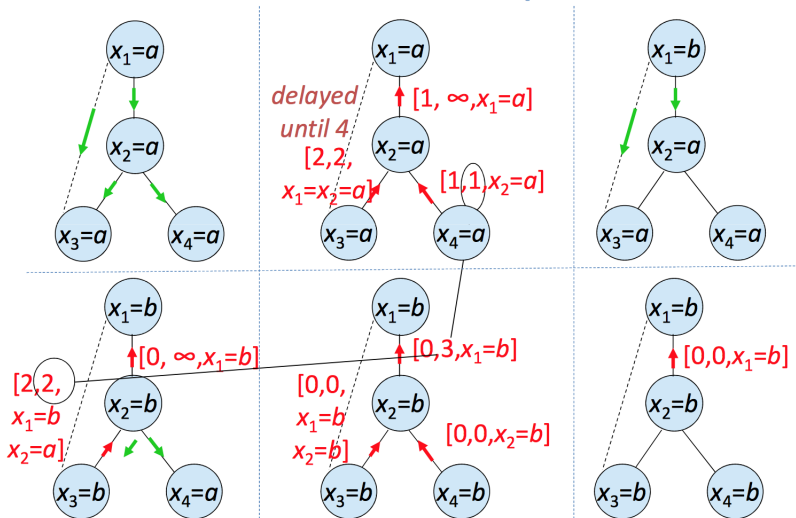
4 binary identical cost functions

$x_i \backslash x_j$	a	b
a	1	2
b	2	0

Constraint graph:



ADOPT : exemple



Threshold

- Lorsqu'un parent envoie une nouvelle valeur à ses voisins inférieurs, ces derniers mettent à jour le contexte. Ils doivent calculer de nouvelles bornes sup et inf pour ce contexte $\Rightarrow$ si le contexte a déjà été rencontré, la valeur de threshold envoyée par le parent permet d'estimer ces bornes.
- Chaque agent maintient une valeur de seuil
- Initialisée à 0 et toujours tel que $LB \leq threshold \leq UB$
- 3 cas possibles de mise à jour :
 - la mise à jour de LB entraîne : $LB > threshold$, $threshold$ est alors augmenté
 - la mise à jour de UB entraîne : $UB < threshold$, $threshold$ est alors diminué
 - une valeur de $threshold$ a été envoyée par le parent (suite à un changement de contexte)

Threshold

- La valeur de *threshold* permet à un parent d'informer un enfant de ses valeurs de borne lorsque ce dernier backtrack à un contexte déjà exploré.
- A partir de sa propre valeur, le parent « distribue » cette valeur de telle façon que la valeur de *threshold* allouée à chaque enfant respecte les bornes du contexte communiquées précédemment par l'enfant au parent.
- Cette méthode évite les calculs redondants tout en limitant la mémoire nécessaire à la recherche de la solution (*polynomial space*).
- La valeur *threshold* est utilisée par un agent pour déterminer quand changer la valeur associée à sa variable : si $LB(d_i) > threshold$, l'agent change sa valeur pour une autre ayant une borne plus petite.

ADOPT : résumé

- ADOPT est un algorithme totalement distribué
- ADOPT calcule une solution optimale
- Espace mémoire polynomial en le nombre de variables
- MAIS dans le pire cas, le nombre de messages échangé est exponentiel en le nombre de variables

ADOPT : utilisation

Code de l'algorithme disponible pour l'enseignement et la recherche : <http://teamcore.usc.edu/dcop/>

Code en Java

Data Sets également disponibles

Plusieurs optimisations d'ADOPT implémentées

DPOP

- DPOP : Distributed Pseudotree Optimization Procedure [PF05]
- Algorithme distribué optimal en communication synchrone
- Résolution basée sur l'inférence (plutôt que la recherche comme pour ADOPT)
- Basé sur les algorithmes somme-produit et DTREE

DPOP

- Organisation des agents en un DFS
- On distingue les liens directs entre les agents d'un niveau n à un niveau $n - 1$ et les liens avec des parents plus éloignés (pseudo-parents) reliant un nœud de niveau n à un niveau $n - k$ avec $k > 1$. (pare exemple x_1 et x_3 dans l'exemple précédent)
- Un nœud X_k de niveau k va envoyer, à son parent X_{k-1} de niveau $k - 1$, les utilités optimales qui peuvent être atteintes par le sous arbre de racine X_k pour chaque valeur possible de X_{k-1}
- A partir des information reçues de ses fils, un nœud X_{k-1} peut déterminer la valeur du sous-arbre dont il est la racine (somme des valeurs reçues) et communiquer cette information à son parent X_{k-2} .

DPOP

- Algorithmes en 3 étapes :
 - Construction du DFS
 - Propagation des valeurs UTIL : des feuilles vers la racine, chaque nœud communique les valeurs d'utilité à son parent
 - Propagation des VALUE : de la racine aux feuilles, chaque nœud communique à ses enfants et pseudo-enfants, la valeur de sa variable

DPOP - phase UTIL

- Des feuilles vers la racine, basée sur la programmation dynamique
- Un nœud reçoit des ses enfants, des valeurs d'utilité (fonctions de coût), les combine et les communique à son parent
- L'utilité pour un nœud dépend de celles de ses enfants mais aussi des valeurs prises par son parent et ses pseudo-parents : calcul d'un hypercube des utilités en fonctions des valeurs prises par le parent et les pseudo-parents.

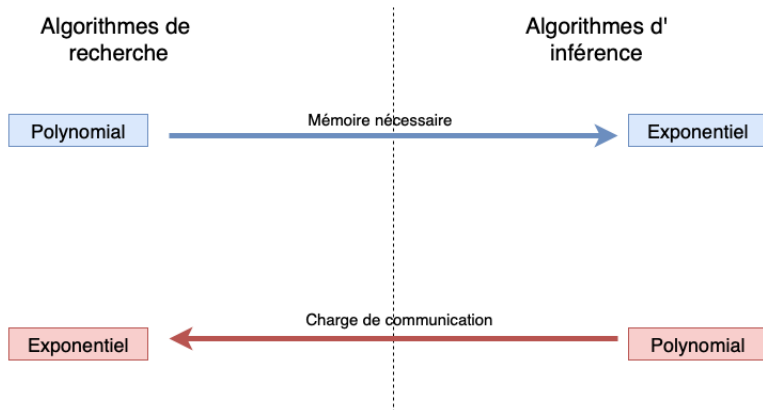
DPOP - phase VALUE

- De la racine vers les feuilles, permet à chaque nœud de déterminer quelle valeur prendre
- Le nœud racine détermine quelle valeur minimise ses coûts et communique cette valeur à ses enfants et pseudo-enfants
- Chaque nœud attend d'avoir reçu les valeurs de son parent et de tous ses pseudo-parents pour déterminer la valeur qui minimise ses coûts et ensuite la communiquer à ses enfants.

DPOP

- Le nombre de messages échangés est linéaire en le nombre de variables (nœuds).
- La taille des messages dans le pire cas est exponentielle en espace en la largeur de l'arbre.
- Remarque : calculer un DFS de largeur minimale est un problème NP-complet.

Algorithmes optimaux - recherche vs. inference



DSA

- DSA : Distributed Stochastic Search Algorithm [ZHA05]
- Algorithme incomplet, complètement distribué, asynchrone basé sur la recherche locale distribuée.
- Chaque agent possède une connaissance locale des valeurs des variables
- Les agents exécutent en parallèle des modifications des valeurs de leurs variables
- Plusieurs stratégies possibles pour la modification des valeurs des variables.

DSA

- Chaque agent exécute la procédure suivante :
 - Choisir aléatoirement une valeur
 - Tant que la terminaison n'est pas atteinte
 - Collecter les nouvelles valeurs des voisins
 - Choisir une nouvelle valeur en fonction de la fonction objectif (minimisation des coûts)
 - Communiquer la nouvelle valeur aux voisins

Autres algorithmes

Algorithmes incomplets

Solutions sous-optimales mais dont le calcul requiert moins de ressources et de communication

- DBA : synchrone, assigne des poids aux contraintes
- MGM : synchrone, propagation de gains unilatéraux → algorithme anytime contrairement à DSA
- ...

Travaux Pratiques

- En TP, nous allons utiliser la bibliothèque *pyDCOP* développée en Python pour résoudre des problèmes formalisés par des DCOP
- <https://pydcop.readthedocs.io/en/latest/index.html>
- Développée par Pierre Rust - Orange - projet sur les *smart objects* connectés

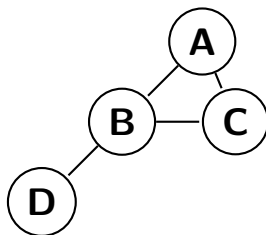
Perspectives de recherche

- Prendre en compte l'incertitude
- Prendre en compte le dynamisme
- Appliquer à des domaines réels : énergie, réseaux de capteurs large échelle, robotique,...

Exercices (extrait de l'examen de 2019)

On considère un ensemble d'agents $\{A, B, C, D\}$ contrôlant chacun une variable x_i qui leur est propre. La variable de chaque agent peut prendre la valeur $\square$ ou $\triangle$.

L'allocation des valeurs à ces variables génère un coût pour le système multi-agents. On souhaite minimiser ce coût qui est égal à la somme des coûts associés aux contraintes entre les agents. Les contraintes entre les agents sont représentées par le graphe suivant :



Exercices (extrait de l'examen de 2019)

Les coûts liés aux contraintes sont donnés par le tableau suivant :

x_i	x_j	coût(A,B)	coût (A,C)	coût (B,C)	coût (B,D)
□	□	5	5	5	3
□	△	8	10	4	8
△	□	20	20	3	10
△	△	3	3	3	3

Par exemple, l'allocation $\{A : \square, B : \square, C : \square, D : \square\}$ génère un coût de 18 ($= 5 + 5 + 5 + 3$).

Exercices (extrait de l'examen de 2019)

- 1 Quelle est la solution optimale à ce problème (allocation des valeurs aux variables) ?
- 2 Formalisez le problème sous forme d'un DCOP.
- 3 Déroulez l'algorithme ADOPT sur ce problème en supposant que les variables sont toutes initialisées à $\square$. Vous veillerez à spécifier l'arbre utilisé et à décrire tous les messages échangés entre les agents.

Références I



Allan R. Leite, Fabrício Enembreck, and Jean-Paul A. Barthès, *Distributed constraint optimization problems : Review and perspectives*, Expert Systems with Applications **41** (2014), no. 11, 5139 – 5157.



Adrian Petcu and Boi Faltings, *A scalable method for multiagent constraint optimization*, Proceedings of the 19th International Joint Conference on Artificial Intelligence (San Francisco, CA, USA), IJCAI'05, Morgan Kaufmann Publishers Inc., 2005, pp. 266–271.



M. Tambe M. Yokoo P.J. Modi, W. Shen, *Adopt : Asynchronous distributed constraint optimization with quality guarantees*, Artificial Intelligence Journal(AIJ) **161** (2005), 149–180.



E. Bowring J.P. Pearce P. Varakantham R.T. Maheswaran, M. Tambe, *Taking dcop to the real world : efficient complete solutions for distributed event scheduling*, 2004.

Références II



Distributed stochastic search and distributed breakout : properties, comparison and applications to constraint optimization problems in sensor networks, Artificial Intelligence **161** (2005), no. 1, 55 – 87, Distributed Constraint Satisfaction.